Requirements Analysis and Specification Document

NightOUT

Version: 0.1
Authors: Riccardo Masarin, Manuel Mannucci, Alessandro Forlani
Project: Software Engineering for Automation (Project A)

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements analysis and specification for **NightOUT**, a mobile-oriented software platform aimed at improving the way users discover nightlife events, interact with other people, and organize their evening experience.

The project addresses the fragmentation of the nightlife experience. Currently, users often need to collect information from different sources to understand which events are available, what type of music or dress code is expected, whether friends are attending, and how to

reserve a spot or join a list. NightOUT aims to centralize these interactions into a single application.

The goal of this document is to describe the application domain, the relevant actors, the main user scenarios, the functional and nonfunctional requirements, and the requirement-level models that specify the expected behavior of the system.

1.2 Scope

NightOUT is intended as a **mobile-oriented web application** for nightlife discovery and organization. The system allows users to browse events, filter them according to relevant criteria, reserve spots, join pregame rooms, interact with events information, and receive notifications related to friends and reservations.

The system also supports **venue managers**, who can manage venues, publish events, approve or define promotions, and access basic statistics related to their events.

The prototype focuses on the following macro-features:

1. **Event discovery and recommendation**
   Users can browse nightlife events and receive suggested events based on music preferences, saved events, social proximity, geographic distance, and booking history.
2. **Event search and filtering**
   Users can filter events by date, location, music genre, venue category, and price or entry condition.
3. **Event popularity dashboard**
   Users can browse events ordered by popularity metrics such as most saved events, most booked events, music genre popularity, and similar events attended.
4. **Social network layer**
   Users can have personal profiles, follow or connect with other users, see participation information, and receive suggestions based on social proximity or shared interests.
5. **Pregame section**
   Users can create or join temporary groups before an event, either in a user-defined location or in a bar available as a pregame venue.
6. **Reservation and ticketing module**
   Users can reserve a spot for an event, join a waiting list, cancel a reservation, and view the reservation status in their profile. The prototype does not manage real payments.
7. **Venue manager features**
   Venue managers can publish events, manage event information, approve promotions, apply changes, and access basic statistics.

1.3 Definitions, Acronyms and Abbreviations

User: registered person who uses NightOUT to discover events, save events, reserve spots, join pregame rooms, and interact with social features.

Guest User: non-authenticated user who may browse a limited subset of public information but cannot reserve spots, join pregame rooms, or access social features.

Venue: physical place such as a club, bar, lounge bar, or live music venue where nightlife events take place.

Venue Manager: verified account responsible for managing one or more venues and publishing events or promotions.

Event: nightlife activity associated with a venue. It includes information such as title, venue, date and time, music genre, dress code, price or entry condition, capacity, and possible promotions.

Reservation: digital record representing a user request to attend an event. A reservation may be pending, confirmed, cancelled, expired, or placed in the waiting list.

Waiting List: list of users who requested a reservation for a full event and may be promoted if a spot becomes available.

Pregame Room: temporary group created before an event, allowing users to coordinate a meeting before attending the event.

Promotion: an offer, discount, or special entry condition associated with a venue or event.

Event Popularity: score/ordering criterion based on indicators such as saved events, reservations, friends attending, genre trends, or similar events attended.

Recommendation Engine: the component responsible for suggesting events to users based on user preferences, social information, distance, and booking history.

1.4 Document Structure

This document is organized as follows:

Section 1 introduces the purpose, scope, definitions, and structure of the document.

Section 2 provides the overall description of the system, including product perspective, scenarios, domain model, user characteristics, product functions, functional requirements, nonfunctional requirements, assumptions, dependencies, and constraints.

Section 3 presents additional requirement-level models, including sequence diagrams, finite state machines, and selected logical specifications.

Section 4 lists the references used in the document.

2. Overall Description

2.1 Product Perspective

NightOUT is a software system designed to centralize several aspects of the nightlife experience into a single platform. The system acts as an intermediary between users, venues, venue managers, events, reservations, social relations, and pregame coordination.

The system is developed as a mobile-oriented web application supported by a Java Spring Boot backend and a relational database. The frontend is optimized for smartphone usage, but it is implemented as a web application to reduce prototype complexity and focus the development effort on the backend architecture, domain model, and core business logic.

NightOUT does not interact with real payment systems or external ticketing providers in the prototype version. Instead, the system simulates the reservation and ticketing process by storing reservation records and showing digital reservation confirmations.

The prototype uses a realistic synthetic dataset generated by the development team. This dataset includes users, venue managers, venues, events, reservations, social relations, pregame rooms, promotions, and notifications. This choice allows the system to demonstrate all relevant scenarios without depending on external APIs or real commercial data.

2.2 Scenarios

The following scenarios describe relevant interactions between the system and the external world. They are written at requirement level and focus on shared phenomena between users, venue managers, events, reservations, and the NightOUT system.

Scenario 1 Discovering an Event

A registered user opens NightOUT to decide where to go during the evening. The system displays a feed of suggested nightlife events. The user filters events by date, location, music genre, venue category, and price or entry condition. The user selects one event and views the event page, which includes venue name, location, date and time, music genre, age target, dress code, price or entry conditions, available promotions, and popularity information.

Relevant actors: User
Relevant system functions: event browsing, filtering, event detail visualization, popularity information.

Scenario 2 Reserving a Spot for an Event

A registered user selects an event and requests a reservation. If the event has available capacity, the system creates a confirmed reservation and displays it in the user profile or ticket section. If the event is full, the system places the user in the waiting list and shows the corresponding reservation status.

Relevant actors: User
Relevant system functions: reservation creation, capacity check, reservation status management, waiting list management.

Scenario 3 Waiting List notification

A user is in the waiting list for a full event. Another user cancels a confirmed reservation. The system detects that a spot is available and promotes the first eligible waiting list reservation to confirmed. The promoted user receives a notification.

Relevant actors: User
Relevant system functions: reservation cancellation, waiting list update, notification.

Scenario 4 Joining a Pregame Room

A registered user opens an event page and browses the pregame rooms associated with that event. The user selects a room with available capacity and joins it. The system updates the room participant list and displays the updated room information. If the room is full, the system prevents the user from joining it.

Relevant actors: User
Relevant system functions: pregame room browsing, join/leave room, participant management.

Scenario 5 Creating a Pregame Room

A registered user creates a pregame room linked to an event. The user specifies a meeting location, meeting time, maximum number of participants, and short description. The system stores the room and makes it visible to other users browsing pregame options for the same event.

Relevant actors: User
Relevant system functions: pregame room creation, event-room association.

Scenario 6 Venue Manager Publishes an Event

A verified venue manager logs into NightOUT and creates a new event for one of their venues. The manager provides event title, date and time, venue, music genre, dress code, price or entry condition, capacity, and possible promotions. The system stores the event and makes it available in the event feed.

Relevant actors: Venue Manager
Relevant system functions: event creation, venue management, promotion management.

Scenario 7 Social Event Discovery

A registered user opens the app and views events that are popular among friends or followed users. The system uses social relations and event participation information to highlight events attended or saved by people connected to the user.

Relevant actors: User
Relevant system functions: social layer, event participation, recommendation, popularity computation.

Scenario 8 Event Notification

A user receives a notification when a relevant event occurs, such as a friend joining an event, a reservation changing status, a waiting list spot becoming available, or an event being updated or cancelled.

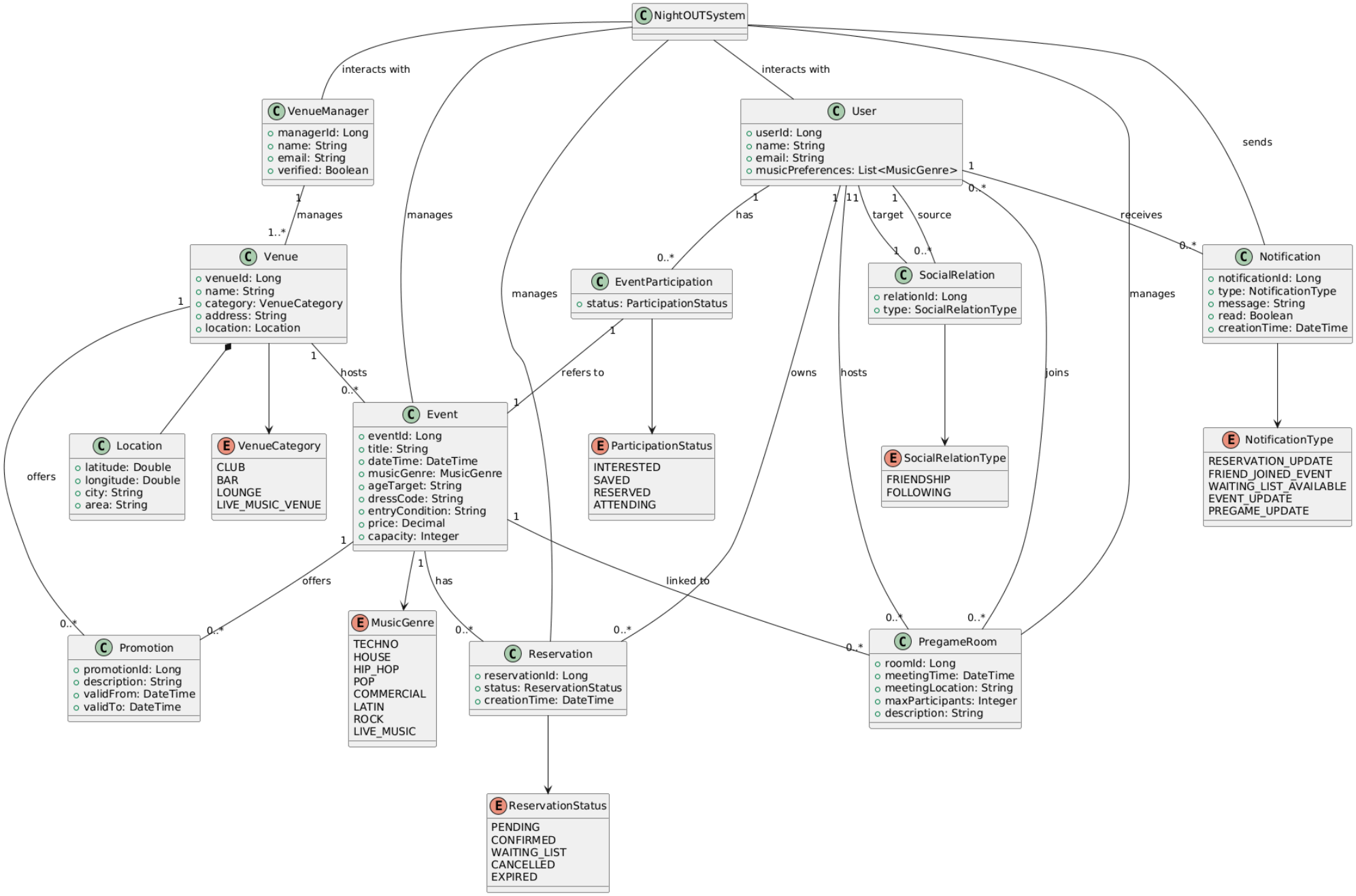
Relevant actors: User
Relevant system functions: notification management, event updates, reservation status updates, social updates.

2.3 Domain Model

The domain model identifies the main concepts of the NightOUT application domain and their relationships. The purpose of the domain model is to describe the problem space, not the final software architecture. This follows the requirement engineering perspective, where the goal is to model the domain, stakeholders, and shared phenomena between the world and the machine.

The main domain entities are:

- User
- Venue Manager
- Venue
- Event
- Reservation
- Pregame Room
- Promotion
- Notification
- Social Relation
- Event Participation
- Music Genre
- Venue Category
- Location


NightOUT - Requirement-Level Domain Model
NightOUTSystem
interacts with
interacts with
sends
VenueManager
+ managerId: Long
+ name: String
+ email: String
+ verified: Boolean
User
+ userId: Long
+ name: String
+ email: String
+ musicPreferences: List<MusicGenre>
manages
manages
manages
manages
receives
Venue
+ venueId: Long
+ name: String
+ category: VenueCategory
+ address: String
+ location: Location
EventParticipation
+ status: ParticipationStatus
has
target
source
SocialRelation
+ relationId: Long
+ type: SocialRelationType
Notification
+ notificationId: Long
+ type: NotificationType
+ message: String
+ read: Boolean
+ creationTime: DateTime
hosts
refers to
owns
hosts
joins
Location
+ latitude: Double
+ longitude: Double
+ city: String
+ area: String
VenueCategory
CLUB
BAR
LOUNGE
LIVE_MUSIC_VENUE
Event
+ eventId: Long
+ title: String
+ dateTime: DateTime
+ musicGenre: MusicGenre
+ ageTarget: String
+ dressCode: String
+ entryCondition: String
+ price: Decimal
+ capacity: Integer
ParticipationStatus
INTERESTED
SAVED
RESERVED
ATTENDING
SocialRelationType
FRIENDSHIP
FOLLOWING
NotificationType
RESERVATION_UPDATE
FRIEND_JOINED_EVENT
WAITING_LIST_AVAILABLE
EVENT_UPDATE
PREGAME_UPDATE
offers
offers
has
linked to
Promotion
+ promotionId: Long
+ description: String
+ validFrom: DateTime
+ validTo: DateTime
MusicGenre
TECHNO
HOUSE
HIP_HOP
POP
COMMERCIAL
LATIN
ROCK
LIVE_MUSIC
Reservation
+ reservationId: Long
+ status: ReservationStatus
+ creationTime: DateTime
PregameRoom
+ roomId: Long
+ meetingTime: DateTime
+ meetingLocation: String
+ maxParticipants: Integer
+ description: String
ReservationStatus
PENDING
CONFIRMED
WAITING_LIST
CANCELLED
EXPIRED

nota: queste parti in UML andranno tolte, servono solo per tenere traccia del codice UML

```
@startuml
title NightOUT - Requirement-Level Domain Model

class NightOUTSystem {
}

class User {
  +userId: Long
  +name: String
  +email: String
  +musicPreferences: List<MusicGenre>
}

class VenueManager {
  +managerId: Long
  +name: String
  +email: String
  +verified: Boolean
}

class Venue {
  +venueId: Long
  +name: String
  +category: VenueCategory
  +address: String
  +location: Location
}

class Event {
  +eventId: Long
  +title: String
  +dateTime: DateTime
  +musicGenre: MusicGenre
  +ageTarget: String
  +dressCode: String
  +entryCondition: String
```

```
  +price: Decimal
  +capacity: Integer
}

class Reservation {
  +reservationId: Long
  +status: ReservationStatus
  +creationTime: DateTime
}

class PregameRoom {
  +roomId: Long
  +meetingTime: DateTime
  +meetingLocation: String
  +maxParticipants: Integer
  +description: String
}

class Promotion {
  +promotionId: Long
  +description: String
  +validFrom: DateTime
  +validTo: DateTime
}

class Notification {
  +notificationId: Long
  +type: NotificationType
  +message: String
  +read: Boolean
  +creationTime: DateTime
}

class SocialRelation {
  +relationId: Long
  +type: SocialRelationType
```

```
}

class EventParticipation {
  +status: ParticipationStatus
}

class Location {
  +latitude: Double
  +longitude: Double
  +city: String
  +area: String
}

enum ReservationStatus {
  PENDING
  CONFIRMED
  WAITING_LIST
  CANCELLED
  EXPIRED
}

enum MusicGenre {
  TECHNO
  HOUSE
  HIP_HOP
  POP
  COMMERCIAL
  LATIN
  ROCK
  LIVE_MUSIC
}

enum VenueCategory {
  CLUB
  BAR
  LOUNGE
```

```
  LIVE_MUSIC_VENUE
}

enum NotificationType {
  RESERVATION_UPDATE
  FRIEND_JOINED_EVENT
  WAITING_LIST_AVAILABLE
  EVENT_UPDATE
  PREGAME_UPDATE
}

enum SocialRelationType {
  FRIENDSHIP
  FOLLOWING
}

enum ParticipationStatus {
  INTERESTED
  SAVED
  RESERVED
  ATTENDING
}

NightOUTSystem -- User : interacts with
NightOUTSystem -- VenueManager : interacts with
NightOUTSystem -- Event : manages
NightOUTSystem -- Reservation : manages
NightOUTSystem -- PregameRoom : manages
NightOUTSystem -- Notification : sends

VenueManager "1" -- "1..*" Venue : manages
Venue "1" -- "0..*" Event : hosts
Event "1" -- "0..*" Reservation : has
User "1" -- "0..*" Reservation : owns

Event "1" -- "0..*" PregameRoom : linked to
```

```
User "1" -- "0..*" PregameRoom : hosts
User "0..*" -- "0..*" PregameRoom : joins

Event "1" -- "0..*" Promotion : offers
Venue "1" -- "0..*" Promotion : offers

User "1" -- "0..*" Notification : receives

User "1" -- "0..*" SocialRelation : source
SocialRelation "1" -- "1" User : target

User "1" -- "0..*" EventParticipation : has
EventParticipation "1" -- "1" Event : refers to

Venue *-- Location
Event --> MusicGenre
Venue --> VenueCategory
Reservation --> ReservationStatus
Notification --> NotificationType
SocialRelation --> SocialRelationType
EventParticipation --> ParticipationStatus

@enduml
```

2.4 User Characteristics

NightOUT has two main categories of users.

Regular Users

Regular users are mainly students, young adults, and nightlife participants who want to discover events, understand what is happening in the city, coordinate with friends, reserve spots, and join pregame groups.

They are expected to use the system mainly from mobile devices. Therefore, usability, clarity of event information, and fast access to the main actions are important aspects.

Regular users need to:

- browse and filter events;
- view event details;
- save events;
- reserve spots or join waiting lists;
- view reservation status;
- join or create pregame rooms;
- see friends' participation information;
- receive notifications.

Venue Managers

Venue managers are verified users responsible for one or more venues. They use the system to publish and manage events, define promotions, update event information, and access basic statistics.

Venue managers need to:

- create events;
- update event information;
- define capacity and entry conditions;
- manage promotions;
- view basic statistics such as saved events, reservations, and popularity indicators.

Administrators

The prototype may include an administrator role for managing system data, verifying venue managers, and moderating content. In the initial prototype, this role can be simplified or simulated.

2.5 Product Functions

The main functions of NightOUT are listed below.

PF-01 Event Discovery

The system allows users to browse a feed of nightlife events.

PF-02 Event Filtering

The system allows users to filter events by date, location, music genre, venue category, and price or entry condition.

PF-03 Event Details

The system allows users to view detailed information about an event, including venue, location, date and time, music genre, age target, dress code, price, entry conditions, promotions, and popularity information.

PF-04 Event Recommendation

The system suggests events based on user preferences, saved events, common friends, geographic distance, and booking history.

PF-05 Event Popularity Dashboard

The system provides an ordered view of events based on popularity metrics such as most saved, most booked, friends attending, and genre trends.

PF-06 Reservation and Ticketing

The system allows users to reserve spots, join waiting lists, cancel reservations, and view reservation status.

PF-07 Pregame Room Management

The system allows users to create, search, join, and leave pregame rooms linked to events.

PF-08 Social Layer

The system allows users to maintain social relations and view event participation information from friends or followed users.

PF-09 Notification Management

The system notifies users about relevant events such as reservation state changes, waiting list availability, friends joining events, and event updates.

PF-10 Venue Manager Event Management

The system allows venue managers to create, update, and manage events and promotions.

2.6 Functional Requirements

The following requirements use the optative form “shall”, consistently with the requirements engineering convention that functional requirements describe what the system must do from the perspective of the software-to-be.

2.6.1 User and Profile Requirements

FR-US-01 — User Registration
The system shall allow a visitor to create a user account by providing the required registration information.

FR-US-02 — User Login
The system shall allow a registered user to log into the system.

FR-US-03 — User Profile Visualization
The system shall allow a registered user to view their profile information.

FR-US-04 — User Preferences
The system shall allow a registered user to define music preferences.

FR-US-05 — Saved Events
The system shall allow a registered user to save an event.

FR-US-06 — View Saved Events
The system shall allow a registered user to view their saved events.

2.6.2 Event Discovery Requirements

FR-EV-01 — Browse Events
The system shall allow users to browse available nightlife events.

FR-EV-02 — View Event Details
The system shall allow users to view the details of a selected event.

FR-EV-03 — Event Information Completeness
For each event, the system shall display at least title, venue name, location, date and time, music genre, dress code, price or entry condition, and capacity information.

FR-EV-04 — Filter Events
The system shall allow users to filter events by date, location, music genre, venue category, and price or entry condition.

FR-EV-05 — Empty Filter Result
When no event satisfies the selected filters, the system shall display an empty result message.

FR-EV-06 — Event Popularity View
The system shall allow users to view events ordered according to selected popularity criteria.

2.6.3 Recommendation Requirements

FR-REC-01 — Recommended Event Feed
The system shall provide a suggested event feed for registered users.

FR-REC-02 — Music-Based Recommendation
The system shall consider user music preferences when generating event recommendations.

FR-REC-03 — Social-Based Recommendation
The system shall consider social proximity and friends' event participation when generating event recommendations.

FR-REC-04 — Distance-Based Recommendation
The system shall consider geographic distance when generating event recommendations.

FR-REC-05 — Booking-History-Based Recommendation
The system shall consider the user's previous reservations when generating event recommendations.

2.6.4 Reservation and Ticketing Requirements

FR-RS-01 — Request Reservation
The system shall allow a registered user to request a reservation for an event.

FR-RS-02 — Confirm Reservation with Available Capacity
When an event has available capacity, the system shall create a confirmed reservation for the requesting user.

FR-RS-03 — Waiting List for Full Events
When an event has no available capacity, the system shall place the requesting user in the waiting list.

FR-RS-04 — View Reservation Status
The system shall allow a registered user to view the status of their reservations.

FR-RS-05 — Cancel Reservation
The system shall allow a registered user to cancel their reservation.

FR-RS-06 — Promote Waiting List Reservation
When a confirmed reservation is cancelled and a waiting list exists for the event, the system shall promote an eligible waiting list reservation to confirmed.

FR-RS-07 — Prevent Duplicate Active Reservations
The system shall not allow a user to have more than one active reservation for the same event.

FR-RS-08 — Reservation Status Values
The system shall represent each reservation using one of the following states: pending, confirmed, waiting list, cancelled, or expired.

FR-RS-09 — Digital Reservation Confirmation
The system shall display a digital reservation confirmation for confirmed reservations.

2.6.5 Pregame Requirements

FR-PR-01 — Create Pregame Room
The system shall allow a registered user to create a pregame room linked to an existing event.

FR-PR-02 — Pregame Room Information
For each pregame room, the system shall store host, linked event, meeting location, meeting time, maximum number of participants, current participants, and description.

FR-PR-03 — Browse Pregame Rooms
The system shall allow users to browse pregame rooms linked to a selected event.

FR-PR-04 — Join Pregame Room
The system shall allow a registered user to join a pregame room if the room has available capacity.

FR-PR-05 — Prevent Joining Full Room
The system shall prevent a user from joining a pregame room whose maximum capacity has been reached.

FR-PR-06 — Leave Pregame Room
The system shall allow a registered user to leave a pregame room previously joined.

FR-PR-07 — Prevent Duplicate Participation
The system shall not allow the same user to join the same pregame room more than once.

2.6.6 Social Layer Requirements

FR-SO-01 — Social Relation Creation
The system shall allow a registered user to create a social relation with another registered user.

FR-SO-02 — View Friends or Followed Users
The system shall allow a registered user to view their social connections.

FR-SO-03 — Event Participation Status
The system shall allow a registered user to indicate interest in or participation in an event.

FR-SO-04 — Friends Attending Event
The system shall allow a registered user to view whether social connections are attending or interested in an event.

FR-SO-05 — Activity Feed
The system shall provide a simple activity feed showing relevant event-related activities among the user’s social connections.

2.6.7 Notification Requirements

FR-NO-01 — Reservation Notification
The system shall notify a user when one of their reservations changes status.

FR-NO-02 — Waiting List Notification
The system shall notify a user when their waiting list reservation is promoted to confirmed.

FR-NO-03 — Friend Event Notification
The system shall notify a user when a social connection joins or saves an event, according to the notification rules of the prototype.

FR-NO-04 — Event Update Notification
The system shall notify users affected by relevant event changes, such as event cancellation or important event information updates.

FR-NO-05 — View Notifications
The system shall allow registered users to view their notifications.

FR-NO-06 — Mark Notification as Read
The system shall allow registered users to mark notifications as read.

2.6.8 Venue Manager Requirements

FR-VM-01 — Venue Manager Login
The system shall allow a verified venue manager to log into the system.

FR-VM-02 — Create Event
The system shall allow a verified venue manager to create an event for a venue they manage.

FR-VM-03 — Update Event
The system shall allow a verified venue manager to update event information for events they manage.

FR-VM-04 — Manage Event Capacity
The system shall allow a verified venue manager to define and update the capacity of an event.

FR-VM-05 — Create Promotion
The system shall allow a verified venue manager to create promotions associated with their venues or events.

FR-VM-06 — View Event Statistics
The system shall allow a verified venue manager to view basic statistics about their events, including number of saved events, number of reservations, and waiting list size.

2.7 Nonfunctional Requirements

Nonfunctional requirements describe user-visible aspects and quality constraints on how system functionalities must be provided. The relevance and priority of nonfunctional requirements depend on the application domain.

2.7.1 Usability

NFR-US-01 — Mobile-Oriented Interface
The system shall provide a responsive user interface optimized for smartphone usage.

NFR-US-02 — Event Reservation Simplicity
The system shall allow a user to request a reservation from the event detail page in no more than three main interaction steps.

NFR-US-03 — Event Information Clarity
The system shall display the main event information in a clear and structured way.

2.7.2 Performance

NFR-PE-01 — Event Feed Response Time
The system shall return the event feed within 2 seconds for the synthetic dataset used in the prototype.

NFR-PE-02 — Filter Response Time
The system shall return filtered event results within 2 seconds for the synthetic dataset used in the prototype.

NFR-PE-03 — Reservation Response Time
The system shall return a reservation status response within 2 seconds under normal prototype execution conditions.

2.7.3 Reliability

NFR-RE-01 — Reservation Consistency
The system shall maintain consistency between event capacity and confirmed reservations.

NFR-RE-02 — No Overbooking
The system shall prevent the number of confirmed reservations for an event from exceeding the event capacity.

NFR-RE-03 — Waiting List Consistency
The system shall maintain a consistent waiting list order for full events.

2.7.4 Robustness

NFR-RO-01 — Invalid Reservation Request
The system shall handle invalid reservation requests without crashing.

NFR-RO-02 — Invalid Pregame Join Request
The system shall handle invalid pregame room join requests without crashing.

NFR-RO-03 — Empty Search Results
The system shall handle empty event search results by displaying an appropriate message.

2.7.5 Maintainability

NFR-MA-01 — Modular Backend Structure
The backend shall be organized into separate components for user management, event management, reservation management, pregame management, notification management, recommendation, and data access.

NFR-MA-02 — Separation of Concerns
The system shall separate presentation logic, application logic, domain logic, and persistence logic.

NFR-MA-03 — Pattern-Based Extensibility
The system design shall support the extension of reservation states, notification types, filtering strategies, popularity strategies, and recommendation strategies.

2.7.6 Security and Privacy

NFR-SP-01 — Authenticated Actions
The system shall require authentication for reservation, pregame participation, social interaction, and venue manager operations.

NFR-SP-02 — User Data Visibility
The system shall prevent non-authenticated users from accessing private user profile information.

NFR-SP-03 — Venue Manager Authorization
The system shall allow venue managers to modify only venues and events they manage.

2.8 Assumptions, Dependencies and Constraints

2.8.1 Domain Assumptions

DA-01 — Synthetic Dataset
The prototype uses a realistic synthetic dataset generated by the development team.

DA-02 — No Real-Time Commercial Data
The prototype does not depend on real-time data from real venues, ticketing platforms, or external event providers.

DA-03 — No Real Payments
The prototype does not process real payments.

DA-04 — Simulated Ticketing
Digital reservation confirmations are simulated and do not represent legally valid tickets.

DA-05 — Verified Venue Managers
Venue managers are assumed to be verified accounts in the prototype dataset.

DA-06 — User Social Relations
Social relations are assumed to be generated or manually inserted in the synthetic dataset.

2.8.2 Dependencies

DEP-01 — Backend Framework
The system depends on a Java Spring Boot backend.

DEP-02 — Frontend Framework
The system depends on a React mobile-first web frontend.

DEP-03 — Database
The system depends on a relational database storing users, events, reservations, pregame rooms, promotions, notifications, and social relations.

DEP-04 — Synthetic Seed Data
The system depends on seed data for demonstration and testing.

2.8.3 Constraints

C-01 — Programming Language
The backend shall be implemented in Java.

C-02 — Backend Architecture
The backend shall expose REST APIs.

C-03 — Frontend Type
The frontend shall be implemented as a responsive web application optimized for mobile usage.

C-04 — No External Payment Integration
The prototype shall not integrate external payment systems.

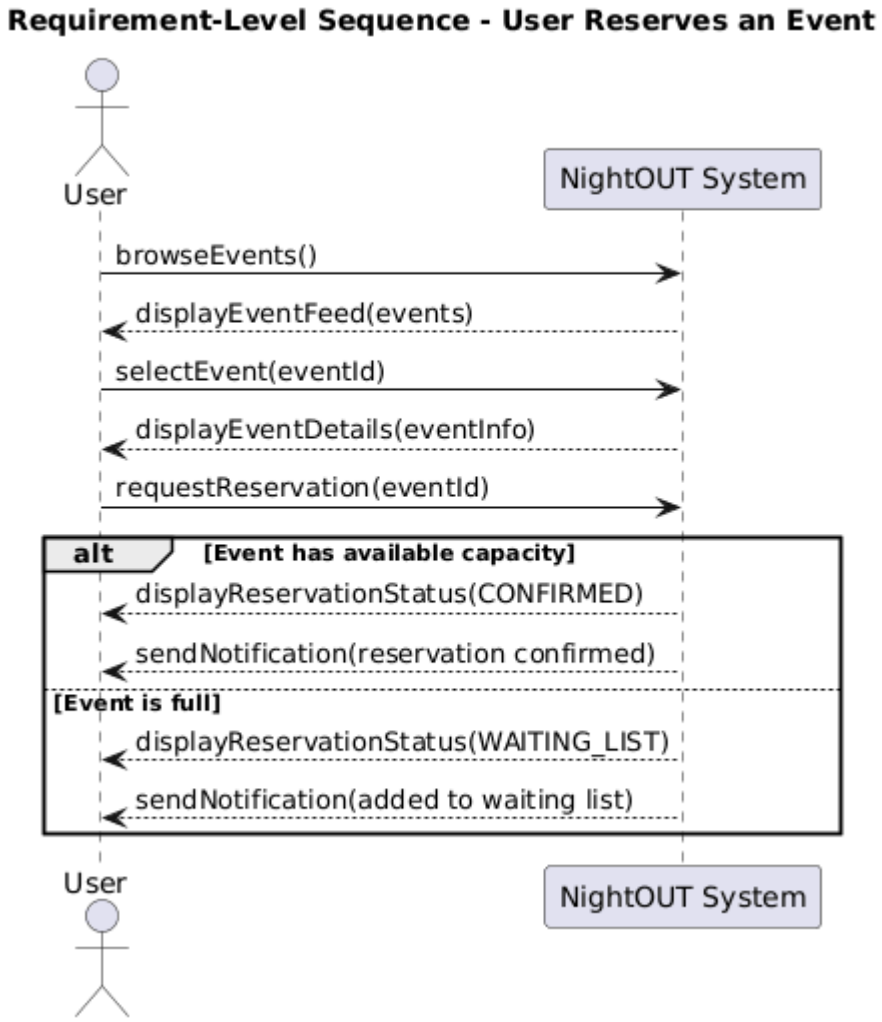
C-05 — No External Event API Requirement
The prototype shall not require external event APIs.

C-06 — Git Repository
All project artifacts shall be released through the project Git repository, as required by the project guideline.

3. Additional Models

3.1 Requirements-Level Sequence Diagrams

3.1.1 User Reserves an Event

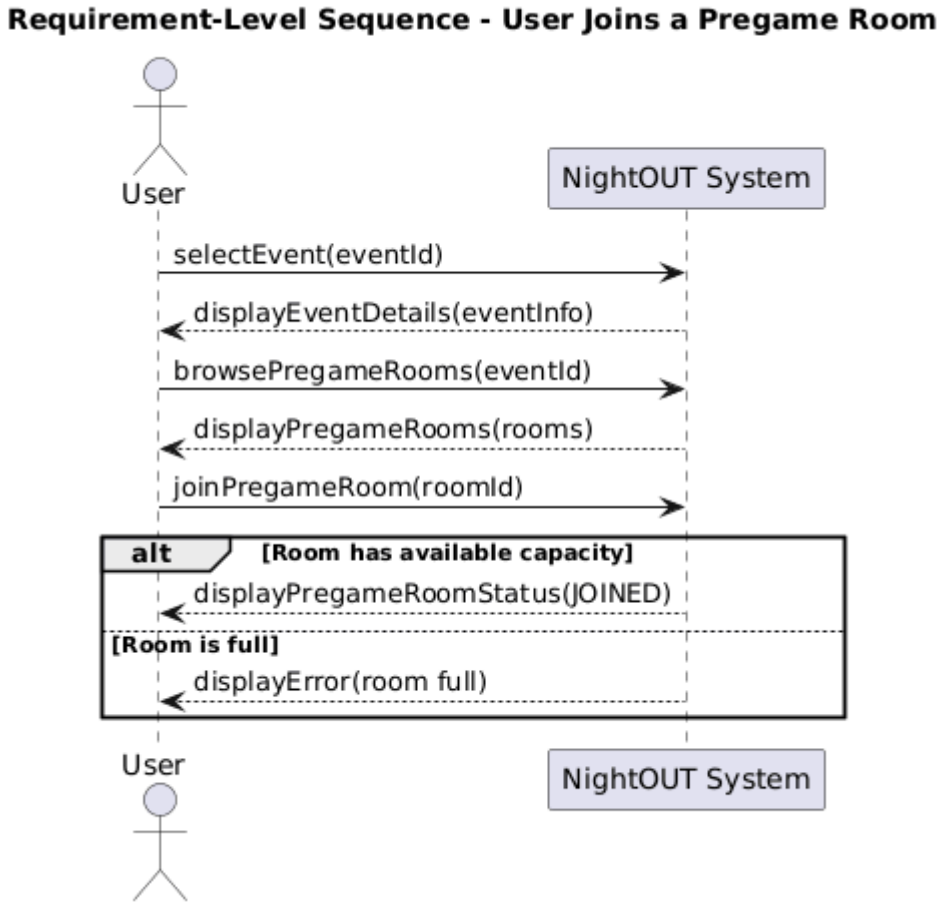


nota: queste parti in UML andranno tolte, servono solo per tenere traccia del codice UML

```
@startuml
title Requirement-Level Sequence - User Reserves an Event

actor User
participant "NightOUT System" as System

User -> System : browseEvents()
System --> User : displayEventFeed(events)

User -> System : selectEvent(eventId)
System --> User : displayEventDetails(eventInfo)
```

```
User -> System : requestReservation(eventId)

alt Event has available capacity
    System --> User : displayReservationStatus(CONFIRMED)
    System --> User : sendNotification(reservation confirmed)
else Event is full
    System --> User : displayReservationStatus(WAITING_LIST)
    System --> User : sendNotification(added to waiting list)
end

@enduml
```

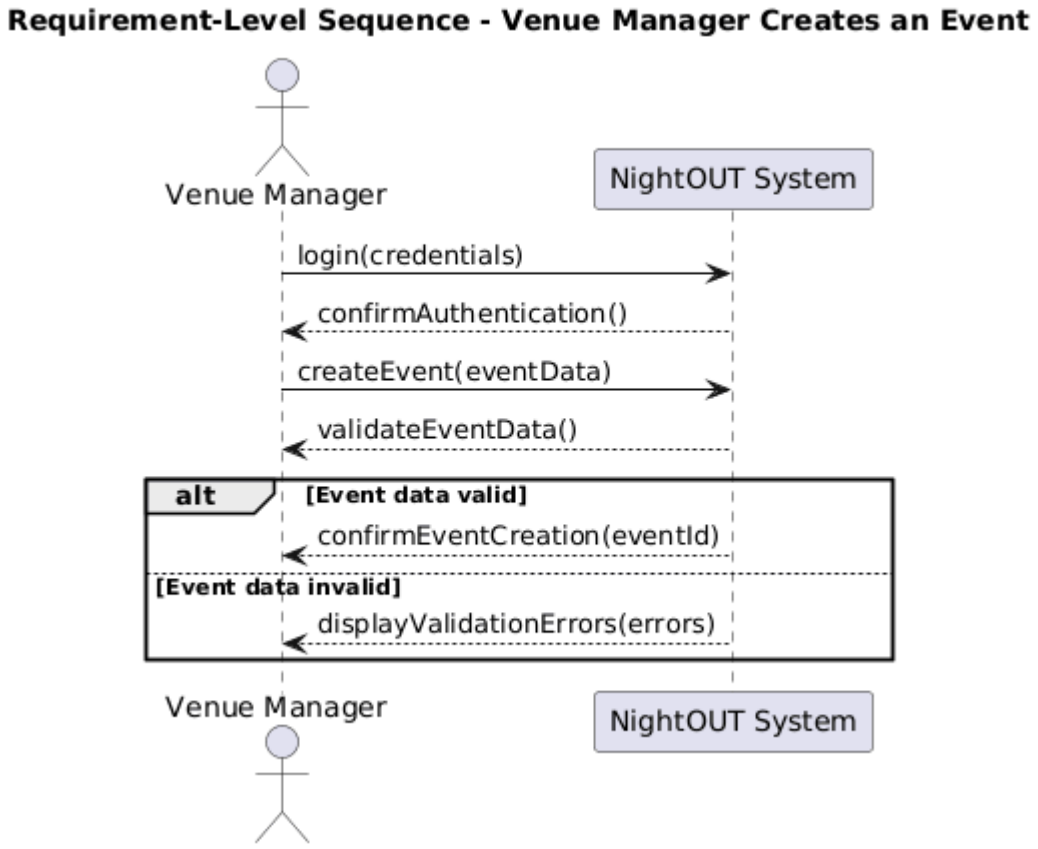
3.1.2 User Joins a Pregame Room




```
@startuml
title Requirement-Level Sequence - User Joins a Pregame Room

actor User
participant "NightOUT System" as System

User -> System : selectEvent(eventId)
System --> User : displayEventDetails(eventInfo)
```

```
User -> System : browsePregameRooms(eventId)
System --> User : displayPregameRooms(rooms)

User -> System : joinPregameRoom(roomId)

alt Room has available capacity
    System --> User : displayPregameRoomStatus(JOINED)
else Room is full
    System --> User : displayError(room full)
end

@enduml
```

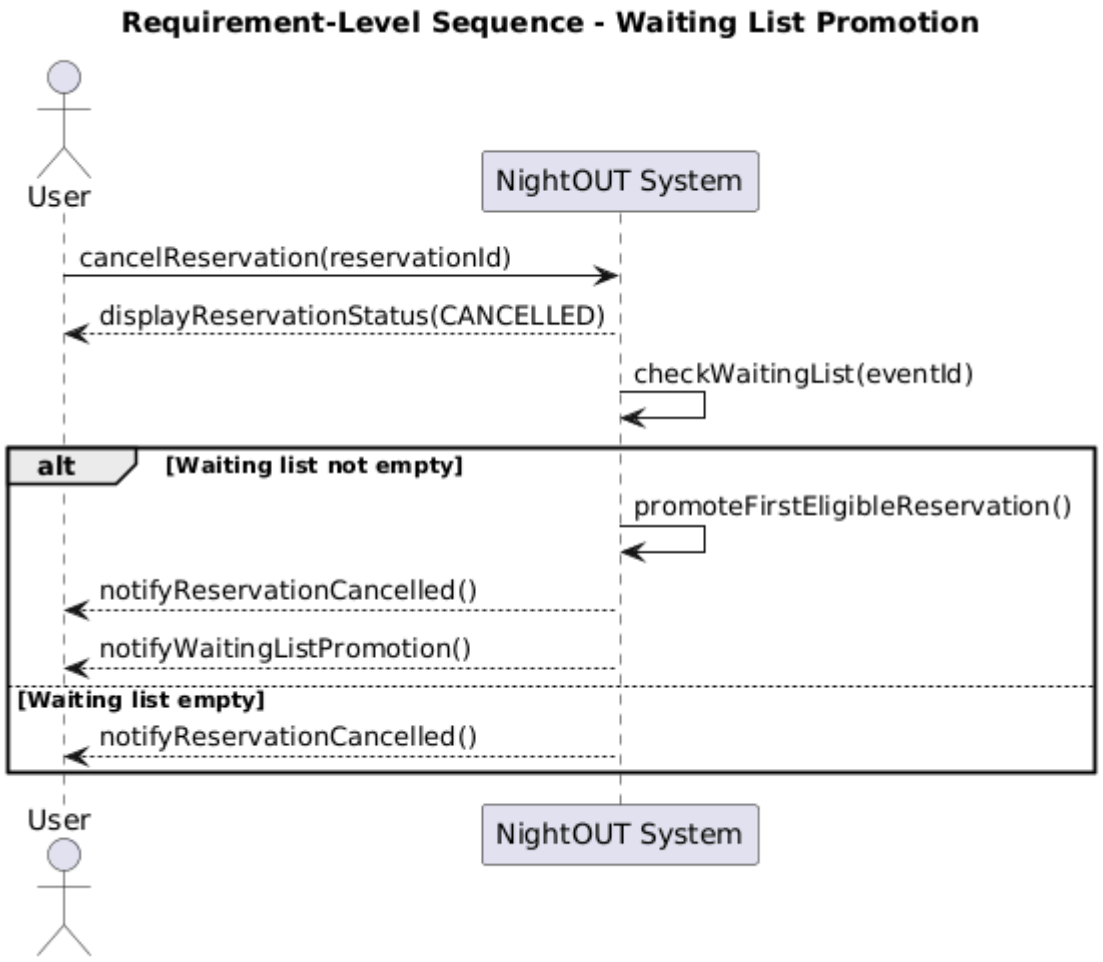
3.1.3 Venue Manager Creates an Event




```
@startuml
title Requirement-Level Sequence - Venue Manager Creates an Event

actor "Venue Manager" as VM
participant "NightOUT System" as System

VM -> System : login(credentials)
System --> VM : confirmAuthentication()
```

```
VM -> System : createEvent(eventData)
System --> VM : validateEventData()

alt Event data valid
    System --> VM : confirmEventCreation(eventId)
else Event data invalid
    System --> VM : displayValidationErrors(errors)
end

@enduml
```

3.1.4 Waiting List Promotion

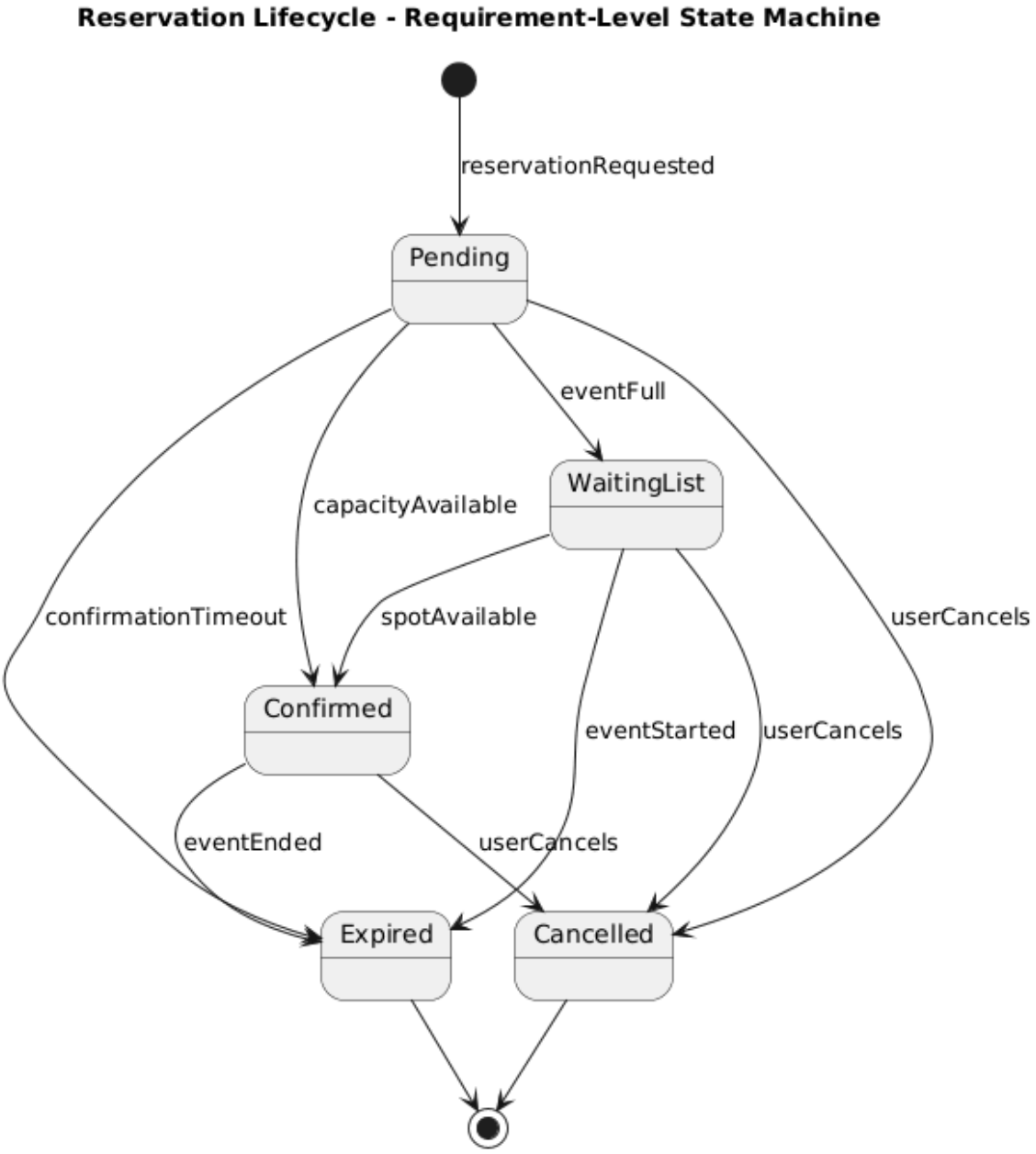



```
@startuml
title Requirement-Level Sequence - Waiting List Promotion

actor User
participant "NightOUT System" as System

User -> System : cancelReservation(reservationId)
System --> User : displayReservationStatus(CANCELLED)
```

```
System -> System : checkWaitingList(eventId)

alt Waiting list not empty
   System -> System : promoteFirstEligibleReservation()
   System --> User : notifyReservationCancelled()
   System --> User : notifyWaitingListPromotion()
else Waiting list empty
   System --> User : notifyReservationCancelled()
end

@enduml
```

3.2 Finite State Machines

3.2.1 Reservation Lifecycle FSM

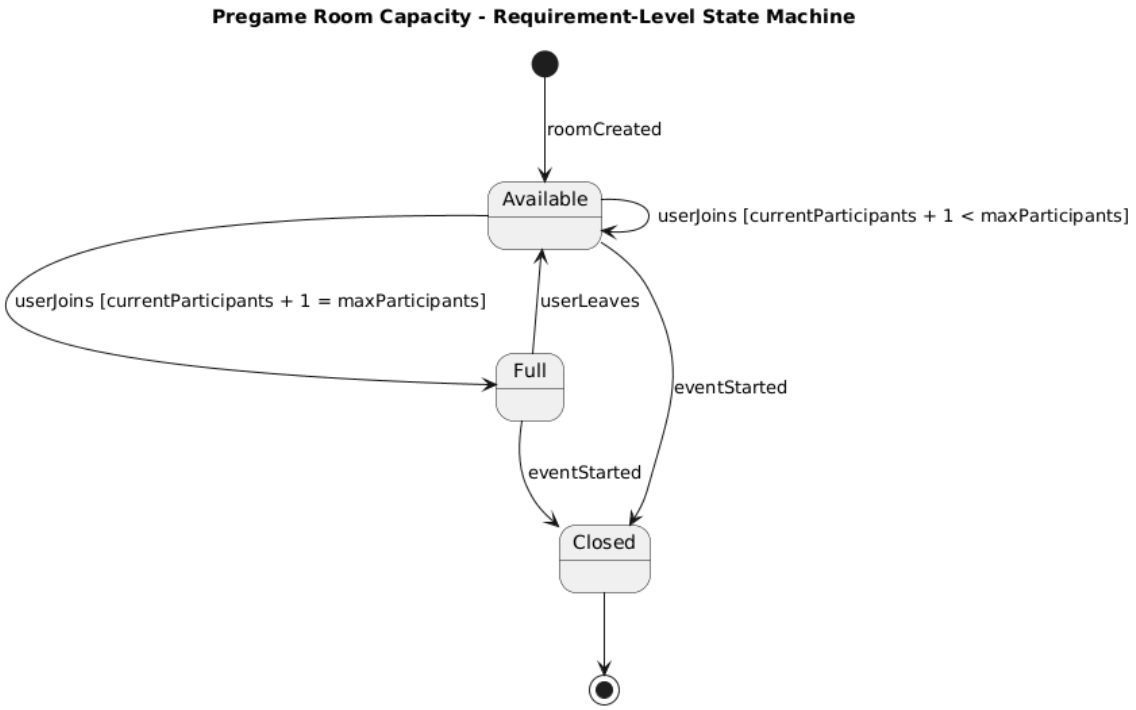



```
@startuml
title Reservation Lifecycle - Requirement-Level State Machine

[*] --> Pending : reservationRequested

Pending --> Confirmed : capacityAvailable
Pending --> WaitingList : eventFull
Pending --> Cancelled : userCancels
Pending --> Expired : confirmationTimeout
```

```
WaitingList --> Confirmed : spotAvailable
WaitingList --> Cancelled : userCancels
WaitingList --> Expired : eventStarted

Confirmed --> Cancelled : userCancels
Confirmed --> Expired : eventEnded

Cancelled --> [*]
Expired --> [*]

@enduml
```

A reservation is created when a user requests to attend an event. Initially, the reservation may be pending. If the event has available capacity, the reservation becomes confirmed. If the event is full, the reservation is placed in waiting list. A user may cancel a pending, waiting list, or confirmed reservation. A waiting list reservation may become confirmed when a spot becomes available. Reservations can expire when the event starts or ends, depending on their current state.

3.2.2 Pregame Room Capacity State Machine




```
@startuml
title Pregame Room Capacity - Requirement-Level State Machine

[*] --> Available : roomCreated

Available --> Available : userJoins [currentParticipants + 1 < maxParticipants]
Available --> Full : userJoins [currentParticipants + 1 = maxParticipants]
```

```
Full --> Available : userLeaves

Available --> Closed : eventStarted
Full --> Closed : eventStarted

Closed --> [*]

@enduml
```

A pregame room is available after creation. Users can join while the number of participants is lower than the maximum capacity. When the room reaches maximum capacity, it becomes full. If a participant leaves a full room, the room becomes available again. When the linked event starts, the pregame room is closed.

3.3 Selected Logical Specifications

3.3.1 Event Filtering

Related requirement: FR-EV-04

Informal requirement:
The system shall allow users to filter events by date, location, music genre, venue category, and price or entry condition.

Specification:

Let `events` be the set of available events, `criteria` the selected filtering criteria, and `res` the set of returned events.

$$events \neq NIL$$

$$\forall e \ (e \in res \Rightarrow e \in events \land satisfies(e, criteria))$$

This means that every returned event must belong to the original event set and must satisfy the selected criteria.

3.3.2 Reservation with Available Capacity

Related requirement: FR-RS-02

Informal requirement:
When an event has available capacity, the system shall create a confirmed reservation for the requesting user.

```
user.isRegistered = true ∧ event.capacity >
event.confirmedReservations

⇒

∃r (
   r.user = user ∧
   r.event = event ∧
   r.status = CONFIRMED
)
```

3.3.3 Reservation for Full Event

Related requirement: FR-RS-03

Informal requirement:
When an event has no available capacity, the system shall place the requesting user in the waiting list.

```
user.isRegistered = true ∧ event.capacity =
event.confirmedReservations

⇒

∃r (
r.user = user ∧
r.event = event ∧
r.status = WAITING_LIST
)
```

3.3.4 No Duplicate Active Reservations

Related requirement: FR-RS-07

Informal requirement:
The system shall not allow a user to have more than one active reservation for the same event.

```
active(r) ⇔
   r.status = CONFIRMED ∨
   r.status = PENDING ∨
   r.status = WAITING_LIST
∀r1,r2 (
   active(r1) ∧ active(r2) ∧
   r1.user = r2.user ∧
   r1.event = r2.event
   ⇒ r1 = r2)
```

3.3.5 Pregame Room Join

Related requirement: FR-PR-04

Informal requirement:
The system shall allow a registered user to join a pregame room if the room is not full and is linked to an existing event.

```
user.isRegistered = true ∧
room.currentParticipants < room.maxParticipants ∧
room.linkedEvent ≠ NIL

⇒

user ∈ room.participants
```

3.3.6 No Duplicate Pregame Participation

Related requirement: FR-PR-07

Informal requirement:
The system shall not allow the same user to join the same pregame room more than once.

```
∀u,r (
countOccurrences(u, r.participants) ≤ 1
)
```

4. References

1. SE4A — Requirements Engineering slides.
2. SE4A — Logic-based specifications slides.
3. SE4A — Verification & Validation and Testing slides.
4. Add Professor Book